Branch Coverage Report

Project: Student Attendance Management System. Activity: Part 3 testing of completed Sprint 1 and Sprint 2 work. Execution date: 6 November 2026. Branch: `feature/testing-feedback`. Base commit: `438845cb25f6`. No Sprint 3 or application feature changes were made.

Measured result

Branch-outcome coverage is 90.64%: 620 of 684 possible branch outcomes were exercised. There are 64 missing outcomes and 36 partial branch locations.

Formula: covered branch outcomes / total possible branch outcomes x 100 = 620 / 684 x 100 = 90.64%.

Coverage.py 7.17.2 measured branches in the final Django regression run with `[run] branch = True`. The 530 discovered tests produced 526 passes and four skips. This is separate from line coverage (94.89%) and the combined coverage.py score (94.19%). An earlier run on the same day, before 30 coverage follow-up tests were added, measured 85.38% (584 of 684). The application code is unchanged between the two runs.

Meaning and scope

A branch outcome is a possible control-flow destination from a branching source line. Coverage.py records executed source-to-destination arcs and reports missing alternatives. This measures branch outcomes, not all execution paths, all input combinations or modified condition/decision coverage. A location may have multiple missing outcomes; partial-branch locations and missing-outcome counts are different quantities.

The measured source is `apps/` and `config/`, including unexecuted importable modules. Exclusions are migrations, test_*.py, tests.py and config/test_runner.py. Third-party packages, the verification harness, manage.py, HTML templates, CSS and JavaScript are outside this Python structural-coverage scope. No application error path was manually excluded; the coverage JSON reports zero excluded lines. Browser correctness is reported separately.

A component with no branch opportunities is reported as N/A. No branch-coverage acceptance threshold was supplied in the attachment; the result is measured, not an invented pass against a percentage target.

Results by component

Component	Outcomes hit	Outcomes total	Coverage	Missing
apps/__init__.py	0	0	N/A	0
apps/academics	114	114	100.00%	0
apps/accounts	187	198	94.44%	11
apps/attendance	184	190	96.84%	6
apps/audit	2	2	100.00%	0
apps/backups	89	132	67.42%	43
apps/notifications	43	46	93.48%	3
config	1	2	50.00%	1

Largest branch gaps

Source file	Missing outcomes	Branch coverage
apps/backups/storage.py	36	33.33%

Source file	Missing outcomes	Branch coverage
apps/attendance/report_views.py	4	90.00%
apps/backups/services.py	4	77.78%
apps/accounts/otp.py	3	94.23%
apps/accounts/permissions.py	3	91.18%
apps/notifications/services.py	3	92.86%
apps/backups/forms.py	2	66.67%
apps/accounts/backends.py	1	50.00%
apps/accounts/credentials.py	1	92.86%
apps/accounts/forms.py	1	91.67%
apps/accounts/services.py	1	95.00%
apps/accounts/views.py	1	92.86%

Windows-specific backup storage is now the dominant gap: 31 of the 36 missing outcomes in `apps/backups/storage.py` are native Windows paths. The follow-up tests covered every previously missing outcome in academics, and the earlier gaps in two-factor pages, snapshot validation and attendance services. Follow-up should exercise meaningful missing alternatives, preserving rollback, authorization and filesystem-safety assertions. A missing branch is not automatically a defect, and a covered branch does not prove every input is safe.

Exact missing outcomes

The raw JSON records `missing_branches` as source/destination line pairs for every file. For example, the largest uncovered file is `apps/backups/storage.py`; its first missing arcs are: 86 -> 87, 86 -> 88, 91 -> -90, 91 -> 92, 146 -> -132, 146 -> 147, 148 -> -132, 148 -> 149. Negative destination numbers denote function/code-object exits rather than source lines. Use the HTML annotations and the source at the recorded revision to interpret each arc.

The HTML source view marks partially executed branch lines and lists unvisited destinations. Full per-file denominators and percentages are in the rerun's `coverage.json` and `coverage-report.txt` in `artifacts/part3/rerun-2026-11-06/`. Keep zero-branch files out of percentage averages; the total reported here uses summed covered outcomes divided by summed possible outcomes.

Evidence and reproduction

Results of the final rerun (test outcomes, test log, coverage JSON and the HTML coverage pages) are kept locally in `artifacts/part3/rerun-2026-11-06/`. Logs and sources from the first run are retained in a separate local supporting-files archive. Neither is included in this submission folder.

Evidence: raw branch arcs and totals, coverage output, instrumented test outcomes and per-file metrics. Open `html/index.html` in `artifacts/part3/rerun-2026-11-06/` for annotated source.

```
OTP_REQUIRED=True VERIFICATION_RESULTS=artifacts/part3/rerun-2026-11-06/regression.json \
  .venv/bin/python -m coverage run --data-file=artifacts/part3/rerun-2026-11-06/.coverage \
  manage.py test --noinput --testrunner verification.test_runner.Runner
.venv/bin/python -m coverage json --data-file=artifacts/part3/rerun-2026-11-06/.coverage \
  -o artifacts/part3/rerun-2026-11-06/coverage.json
.venv/bin/python -m coverage html --data-file=artifacts/part3/rerun-2026-11-06/.coverage \
  -d artifacts/part3/rerun-2026-11-06/html
```

The committed `.coveragerc` already enables branch measurement. Run in the isolated environment described by the Unit Test Report. Browser and performance executions are not included in the measured arc totals. Native Windows branches, deployment behavior and the outstanding requirements findings remain separate validation work.